Pattern Printing

DEFINITION

Pattern printing is the practice of using nested loops to arrange characters into a shape — a square, a triangle, a pyramid, a diamond. The shape itself isn't the point: this topic exists to train you to see how the outer loop (row), the inner loop (column), and the relationship between them (i and j) control every character that gets printed. Every hard DSA loop problem later builds on this exact instinct.

The Golden Rule

Every pattern has two loops: the outer loop decides which row you're on, and the inner loop decides what gets printed in that row. Before writing any code, answer three questions on paper first:

Question	Decides
How many rows?	Range of the outer loop (i)
How many things per row?	Range of the inner loop (j) — often depends on i
What gets printed?	* , a number, a letter, or a space — depends on i and/or j

★ NOTE

- Always write the outer loop first and ask 'how many rows are there?' — that number becomes your outer loop's limit.
- Then, for ONE row, work out the inner loop by hand before generalizing it with i.
- If the inner loop's limit changes with i (like 1, 2, 3...), that's your biggest clue it's a triangle-family pattern, not a square.

1. Square / Rectangle Pattern

The simplest pattern: every row prints the same number of columns, so the inner loop's limit never depends on i.

C++

```
int n = 4;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        cout << "* ";
    }
    cout << endl;
}
```

Output

```
* * * *
* * * *
* * * *
* * * *
```

★ NOTE

- Inner loop condition is j <= n — fixed, it never looks at i.
- This is the pattern every other one deviates from: as soon as the inner limit starts depending on i, you get a triangle instead of a rectangle.

2. Right-Angled Triangle (Half Pyramid)

Now the inner loop's limit depends on the current row i — row 1 prints 1 star, row 2 prints 2 stars, and so on.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        cout << "* ";
    }
    cout << endl;
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Swap j <= i for j <= (n - i + 1) and you get the mirror image: a triangle that shrinks as it goes down.

3. Number Triangle

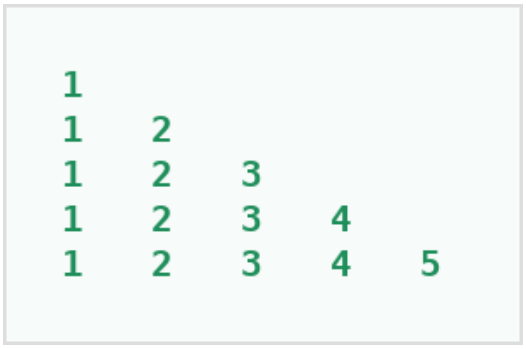
Same shape as above, but instead of printing a fixed '*', we print the value of the loop variable itself.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        cout << j << " ";
    }
    cout << endl;
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```



★ NOTE

- Print cout << i instead of cout << j to get repeated row numbers: 1 / 2 2 / 3 3 3 / 4 4 4 4 — same loops, only the printed value changes.

4. Inverted Right Triangle

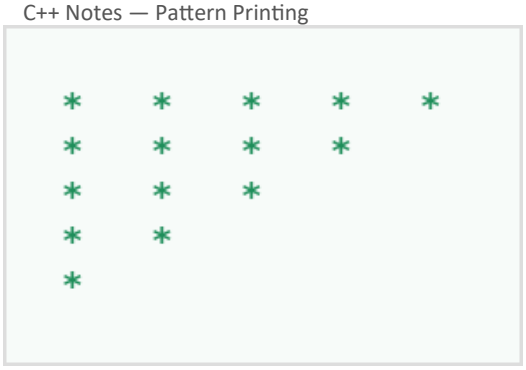
Flip the row count so it decreases: start the outer loop at n and count down, or keep it counting up and invert the inner limit.

C++

```
int n = 5;
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) {
        cout << "* ";
    }
    cout << endl;
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```



5. Pyramid (Centered Triangle)

A true pyramid needs leading spaces to center each row, on top of the stars. Each row i needs (n - i) spaces followed by (2*i - 1) stars — work this out on paper for n = 4 before trusting the formula.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++) {
        cout << " ";
    }
    for (int j = 1; j <= 2 * i - 1; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Output

```
    *
   ***
  *****
 *******
*********
```

★ NOTE

- It's normal for a single row to need two separate inner loops back to back — one for spaces, one for the visible characters.
- Space count shrinks as i grows (n - i); star count grows as i grows (2*i - 1) — that's what makes the sides slant inward at the top.

6. Inverted Pyramid

Same two-loop idea, but spaces grow while stars shrink as i increases.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i - 1; j++) {
        cout << " ";
    }
    for (int j = 1; j <= 2 * (n - i) + 1; j++) {
        cout << "*";
    }
    cout << endl;
}
```

Output

```
*********
 *******
  *****
   ***
    *
```

7. Diamond

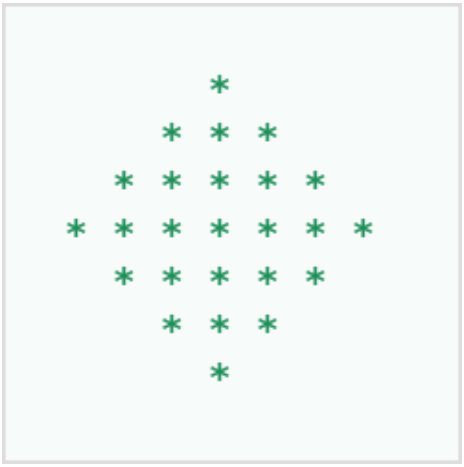
A diamond is nothing new — it's a pyramid immediately followed by an inverted pyramid that starts one row smaller, stitched together.

C++

```
int n = 4;
// top half (pyramid)
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n - i; j++) cout << " ";
    for (int j = 1; j <= 2 * i - 1; j++) cout << "*";
    cout << endl;
}
// bottom half (inverted pyramid, one row smaller)
for (int i = n - 1; i >= 1; i--) {
    for (int j = 1; j <= n - i; j++) cout << " ";
    for (int j = 1; j <= 2 * i - 1; j++) cout << "*";
    cout << endl;
}
```

Output

```
    *
   ***
  *****
*******
  *****
   ***
    *
```



★ **NOTE**

- Notice the bottom half is the exact same pyramid loop, just with i counting down from n-1 instead of up to n.
- Most 'complex-looking' patterns (diamonds, hourglasses, butterflies) are two familiar patterns glued together — spot the two halves before coding.

8. Floyd's Triangle (Continuous Numbers)

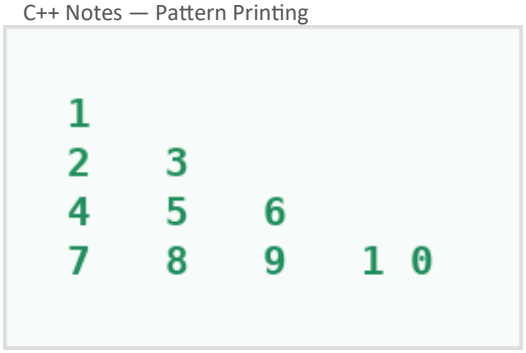
Instead of restarting the count every row, a single counter keeps increasing across the entire pattern.

C++

```
int n = 4, num = 1;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        cout << num << " ";
        num++;
    }
    cout << endl;
}
```

Output

```
1
2 3
4 5 6
7 8 9 10
```



★ NOTE

- The counter (num) lives outside both loops and is declared once — it must NOT reset inside the outer loop, or every row would restart from 1.

9. Character Pattern

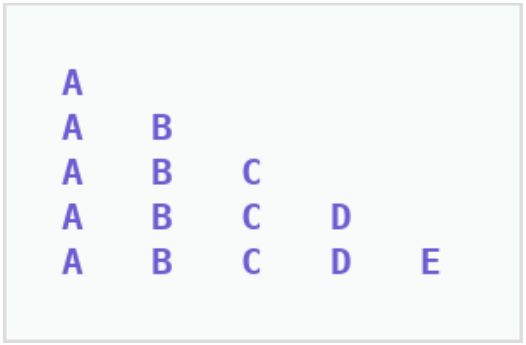
Characters work the same as numbers, just add 'A' (or 'a') to a numeric offset to convert it — this trick shows up constantly.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        char ch = 'A' + (j - 1);
        cout << ch << " ";
    }
    cout << endl;
}
```

Output

```
A
A B
A B C
A B C D
A B C D E
```



10. Hollow Square

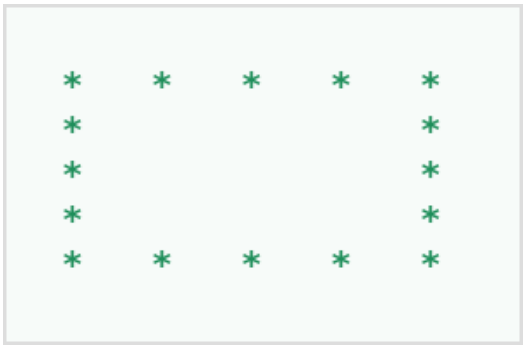
A classic interview favorite. The border cells (first row, last row, first column, last column) get a '*'; everything else gets a space. Notice this needs an if-else INSIDE the inner loop instead of a separate loop for spaces.

C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
        if (i == 1 || i == n || j == 1 || j == n) {
            cout << "* ";
        } else {
            cout << "  ";
        }
    }
    cout << endl;
}
```

Output

```
* * * * *
*       *
*       *
*       *
* * * * *
```



★ NOTE

- This is the first pattern where the DECISION to print '*' or ' ' happens inside a single inner loop via a condition, rather than two separate loops.
- The border condition (i == 1 || i == n || j == 1 || j == n) is worth memorizing — it appears in hollow rectangles, borders, and matrix-boundary problems constantly.

11. Hollow Right Triangle

Same hollow idea, but applied to a triangle instead of a square: only the first row, the last row, and the two slanted/vertical edges get a star.

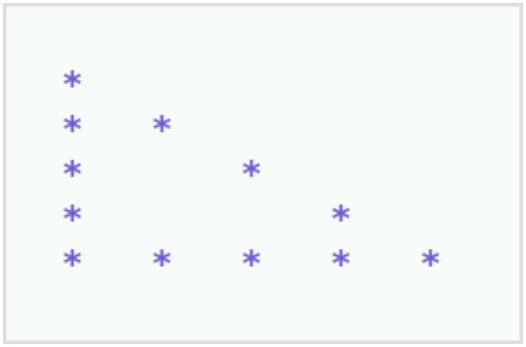
C++

```
int n = 5;
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) {
        if (j == 1 || j == i || i == n) {
            cout << "* ";
        } else {
            cout << "  ";
        }
    }
```

```
    cout << endl;
}
```

Output

```
*
* *
*   *
*     *
* * * * *
```



12. Butterfly Pattern

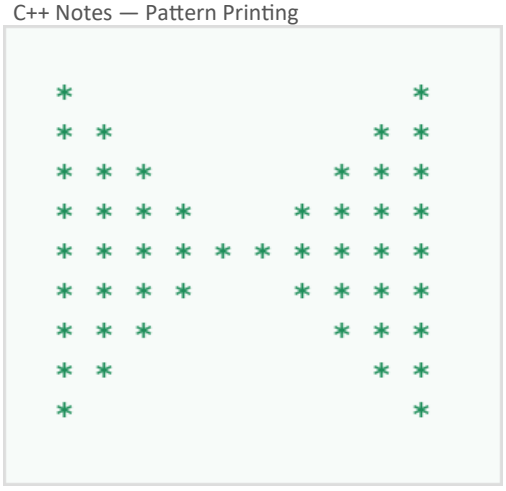
Two triangles facing each other, mirrored left-right, stitched top half and bottom half — same building blocks as the diamond, just wider because there's no gap-closing to the centre point.

C++

```
int n = 5;
// top half
for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= i; j++) cout << "*";
    for (int j = 1; j <= 2 * (n - i); j++) cout << " ";
    for (int j = 1; j <= i; j++) cout << "*";
    cout << endl;
}
// bottom half (mirror of the top, i counts back down)
for (int i = n; i >= 1; i--) {
    for (int j = 1; j <= i; j++) cout << "*";
    for (int j = 1; j <= 2 * (n - i); j++) cout << " ";
    for (int j = 1; j <= i; j++) cout << "*";
    cout << endl;
}
```

Output

```
*         *
**       **
***     ***
****   ****
**********
****   ****
***     ***
**       **
*         *
```



★ NOTE

- Left wing, middle gap, right wing — three separate inner loops in a row, same row.
- The bottom half is just the top half's outer loop run backwards, exactly like the diamond's second half.

13. Pascal's Triangle

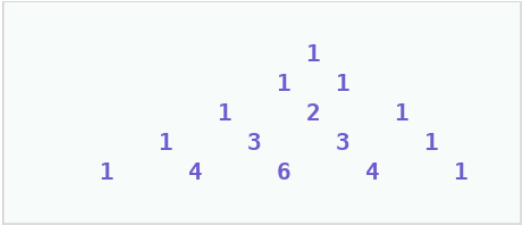
Each entry is nCr — the number of ways to choose r items from n. Rather than compute factorials, build each row from the previous one: multiply by (row - col) and divide by col as you go, which avoids overflow for longer rows.

C++

```
int n = 5;
for (int i = 0; i < n; i++) {
    for (int s = 0; s < n - i - 1; s++) cout << "  ";
    long long val = 1;
    for (int j = 0; j <= i; j++) {
        cout << val << "  ";
        val = val * (i - j) / (j + 1);
    }
    cout << endl;
}
```

Output

```
        1
      1  1
    1  2  1
  1  3  3  1
1  4  6  4  1
```



★ NOTE

- The value printed isn't a function of i and j alone — it's built iteratively as you move across the row, which is the bridge from simple loop patterns into DP-style row-by-row construction.

14. Zigzag Pattern

A 3-row wave built from a single running condition. Track which 'peak' each column belongs to and alternate the row a star lands on.

C++

```
int n = 12;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < n; j++) {
        if (i == 0 && j % 4 == 0)
            cout << "*";
        else if (i == 1 && (j % 4 == 1 || j % 4 == 3))
            cout << "*";
        else if (i == 2 && j % 4 == 2)
            cout << "*";
        else
            cout << " ";
    }
    cout << endl;
}
```

Output

```
*   *   *   *
 * * * * * *
  *   *   *
```

★ NOTE

- It's the first pattern where the printed character depends on (i, j) through a modulo condition rather than a clean range — good practice for reading tricky conditions before you hit them in a real problem.

ROUGH WORK

ROUGH WORK

ROUGH WORK

End of Notes — Pattern Printing

NextGen Data Minds